Section 3: Arrays

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 21: Section Intro, Arrays

SLIDE 22: Array Fundamentals and In-Place Operations

SLIDE 23: Prefix Sums

SLIDE 24: Subarray Patterns and Kadane's Algorithm

SLIDE 24B: Kadane's Algorithm Step-by-Step Trace

SLIDE 25: Worked Example: Subarray Sum Equals K

SLIDE 26: Worked Example: Rotate Array

SLIDE 27: Wrong vs Right: Array Mistakes

SLIDE 28: How to Present Array Solutions in an Interview

SLIDE 29: Interview Practice: Arrays

SLIDE 30: Section Cheat Sheet

SLIDE 21: Section Intro, Arrays

Section 3: Arrays

Arrays are the most frequently tested data structure in coding interviews. Nearly every problem either operates on an array directly or converts its input into one before processing. If you master array techniques, you have a foundation for at least half of all interview problems.

An array stores elements in contiguous memory, which gives you O(1) access to any element by index. That single property, constant-time random access, is what makes arrays the backbone of nearly every other technique in this course.

What We'll Cover

1. Array fundamentals and operation costs
2. In-place operations with the read/write pointer pattern
3. Prefix sums for O(1) range queries
4. Subarray vs subsequence distinction
5. Kadane's algorithm for maximum subarray sum
6. Worked examples: Subarray Sum Equals K, Rotate Array

Where Arrays Connect to Future Sections

Future Section	How Arrays Connect
Two Pointers (Section 6)	Operates on arrays using opposite-direction and same-direction traversal
Sliding Window (Section 7)	Maintains a moving window over an array or string
Binary Search (Section 8)	Operates on sorted arrays, halving the search space each step
Sorting (Section 9)	Rearranges arrays as preprocessing to unlock simpler solutions

Every technique from Section 6 through Section 9 assumes you are comfortable with array operation costs, in-place manipulation, and prefix sums. This section builds that prerequisite.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 22: Array Fundamentals and In-Place Operations

Contiguous Memory and O(1) Access

An array allocates a single block of memory. Element at index i sits at address (base + i * elementSize). The CPU calculates this in constant time, which is why arr[i] is O(1) regardless of array size.

This is the fundamental trade-off: arrays give you instant access to any position, but inserting or deleting in the middle requires shifting every element after that position.

Operation Cost Table

Operation	Time Complexity	Why
Access by index	O(1)	Direct address calculation
Search (unsorted)	O(n)	Must check every element
Search (sorted)	O(log n)	Binary search (Section 8)
Append to end	O(1) amortized	Occasional resize costs O(n), averaged over n operations
Insert at index i	O(n)	Shift all elements from i onward
Delete at index i	O(n)	Shift all elements from i onward

In-Place Operations

“In-place” means O(1) auxiliary space. You modify the array itself rather than creating a new one. Two common in-place patterns:

Swap elements: exchange values at two indices without extra storage.

Read/write pointer: the read pointer scans every element, the write pointer tracks where the next valid element should go. Elements before the write pointer are the result. Elements at or after it are don’t-care values.

Worked Example: Remove Duplicates from Sorted Array (LeetCode 26)

Given a sorted array, remove duplicates in place and return the new length.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;

    int write = 1;
```

```
    for (int read = 1; read < nums.length; read++) {
        if (nums[read] != nums[read - 1]) {
            nums[write] = nums[read];
            write++;
        }
    }

    return write;
}
```

Trace through [1, 1, 2, 2, 3]:

Step	read	nums[read]	Condition	Action	Array State	write
Start	-	-	-	-	[1, 1, 2, 2, 3]	1
1	1	1	1 == 1, skip	-	[1, 1, 2, 2, 3]	1
2	2	2	2 != 1, copy	nums[1] = 2	[1, 2, 2, 2, 3]	2
3	3	2	2 == 2, skip	-	[1, 2, 2, 2, 3]	2
4	4	3	3 != 2, copy	nums[2] = 3	[1, 2, 3, 2, 3]	3

Result: first 3 elements [1, 2, 3] are the deduplicated array. Return value: 3.

Time O(n), space O(1). This read/write pointer pattern reappears in Section 6 (Two Pointers) as the same-direction variant.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 23: Prefix Sums

The Problem Prefix Sums Solve

Suppose you need the sum of elements from index 2 to index 5 in an array. Without preprocessing, you iterate from index 2 to 5 and add them up: O(n) per query. If you need to answer 1000 different range sum queries on the same array, that is 1000 * O(n) = O(n * q) total work.

Prefix sums let you answer every range sum query in O(1) after a single O(n) preprocessing pass. Total work: O(n + q) instead of O(n * q).

How to Build a Prefix Sum Array

The prefix sum array stores cumulative sums. prefix[i] holds the sum of all elements from index 0 through index i.

```
int[] buildPrefixSum(int[] nums) {
    int[] prefix = new int[nums.length];
    prefix[0] = nums[0];

    for (int i = 1; i < nums.length; i++) {
        prefix[i] = prefix[i - 1] + nums[i];
```

```
    }

    return prefix;
}
```

Trace for nums = [2, 4, 1, 7, 3]:

Index	0	1	2	3	4
nums	2	4	1	7	3
prefix	2	6	7	14	17

How to Query a Range Sum

Sum from index l to index r:

- If l == 0: answer is prefix[r]
- If l > 0: answer is prefix[r] - prefix[l - 1]

```
int rangeSum(int[] prefix, int l, int r) {
    if (l == 0) return prefix[r];
    return prefix[r] - prefix[l - 1];
}
```

Example queries on nums = [2, 4, 1, 7, 3]:

Query	Formula	Calculation	Result
sum(0, 4)	prefix[4]	17	17 (entire array)
sum(1, 3)	prefix[3] - prefix[0]	14 - 2	12 (4 + 1 + 7)
sum(2, 4)	prefix[4] - prefix[1]	17 - 6	11 (1 + 7 + 3)
sum(2, 2)	prefix[2] - prefix[1]	7 - 6	1 (single element)

When to Recognize Prefix Sums

Look for these signals in the problem statement: “subarray sum”, “range sum”, “cumulative”, or any problem that computes sums over many different subarrays of the same array.

Interview Tip: When you see “subarray sum equals K” or “count subarrays with sum equal to target”, prefix sums combined with a HashMap is almost always the O(n) approach. We solve exactly this problem in Slide 25.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 24: Subarray Patterns and Kadane’s Algorithm

Subarray vs Subsequence

This distinction appears in problem statements constantly, and confusing them leads to applying the wrong technique.

Property	Subarray	Subsequence
Contiguous	Yes, elements must be adjacent	No, elements can be non-adjacent
Order preserved	Yes	Yes
Example from [1,2,3,4]	[2,3,4] is valid	[1,3,4] is valid
Invalid example	[1,3] is NOT a subarray	All subarrays are also subsequences
Techniques	Prefix sum, sliding window, Kadane's	DP, typically covered in C6

When a problem says "subarray", it means contiguous. This is good news: contiguous constraints enable efficient techniques.

Three Core Subarray Techniques

Technique	What It Solves	Time	Section
Prefix Sum	Range sum queries, count subarrays with given sum	O(n) preprocess, O(1) per query	This section
Sliding Window	Longest/shortest subarray satisfying a condition	O(n)	Section 7
Kadane's Algorithm	Maximum (or minimum) sum contiguous subarray	O(n)	This section

Kadane's Algorithm

At each position, you make one decision: extend the current subarray, or start a new one here. The recurrence:

currentMax = max(nums[i], currentMax + nums[i])

If adding nums[i] to the running sum makes it larger than nums[i] alone, extend. Otherwise, the previous subarray was dragging the sum down, so start fresh from nums[i].

```
public int maxSubArray(int[] nums) {
    int currentMax = nums[0];
    int globalMax = nums[0];

    for (int i = 1; i < nums.length; i++) {
        currentMax = Math.max(nums[i], currentMax + nums[i]);
        globalMax = Math.max(globalMax, currentMax);
    }

    return globalMax;
}
```

Critical detail: initialize both currentMax and globalMax to nums[0], not to 0. Initializing to 0 fails on all-negative arrays. For [-3, -2, -1], the correct answer is -1, but initializing to 0 would return 0 (an empty subarray sum that does not exist in the problem).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 24B: Kadane's Algorithm Step-by-Step Trace

Trace through [-2, 1, -3, 4, -1, 2, 1, -5, 4]:

i	nums[i]	currentMax + nums[i]	nums[i] alone	currentMax (max of two)	globalMax
0	-2	(init)	(init)	-2	-2
1	1	-2 + 1 = -1	1	1 (start fresh)	1
2	-3	1 + (-3) = -2	-3	-2 (extend)	1
3	4	-2 + 4 = 2	4	4 (start fresh)	4
4	-1	4 + (-1) = 3	-1	3 (extend)	4
5	2	3 + 2 = 5	2	5 (extend)	5
6	1	5 + 1 = 6	1	6 (extend)	6
7	-5	6 + (-5) = 1	-5	1 (extend)	6
8	4	1 + 4 = 5	4	5 (extend)	6

Result: maximum subarray sum is 6, from subarray [4, -1, 2, 1] (indices 3 through 6).

Reading the Trace

At index 1, starting fresh wins because the previous running sum (-2) was negative, meaning it would only drag down any future subarray. At index 3, the same thing happens: the running sum of -2 is again negative, so a fresh start from 4 is better. These are the decision points that make Kadane's work.

At index 7, even though -5 hurts the running sum, extending is still better than starting fresh at -5 alone. The algorithm correctly holds on because the accumulated sum (1) is still positive, meaning there is still a chance that future positive elements will push globalMax higher (and they do, at index 8).

When to Recognize Kadane's

Problem statement signals: "maximum sum contiguous subarray", "maximum subarray", "largest sum of consecutive elements." If you see these phrases and the constraint is n up to 10^5 or 10^6, Kadane's O(n) is the expected solution. The brute force O(n^2) that checks every subarray will not pass within time limits at that scale.

Time O(n), space O(1). One of the most elegant algorithms in interview prep.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 25: Worked Example: Subarray Sum Equals K

Problem: Subarray Sum Equals K (LeetCode 560)

Given an integer array nums and an integer k, return the total number of contiguous subarrays whose sum equals k.

Step 1: Understand

The array can contain negative numbers, so the sliding window technique does not apply (adding elements can decrease the sum, violating the monotonic property that sliding window requires). We need a different approach.

Step 2: Brute Force

Check every possible subarray. For each starting index i, compute the running sum from i to every ending index j.

```
public int subarraySum(int[] nums, int k) {
    int count = 0;

    for (int i = 0; i < nums.length; i++) {
        int sum = 0;
        for (int j = i; j < nums.length; j++) {
            sum += nums[j];
            if (sum == k) count++;
        }
    }

    return count;
}
```

Time O(n^2), space O(1).

Step 3: Optimal with Prefix Sum + HashMap

The key insight: if prefix[j] - prefix[i] == k, then the subarray from index i+1 to j sums to k. Instead of scanning backward for every j, store how many times each prefix sum has occurred in a HashMap and look up (currentPrefix - k) in O(1).

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);

    int currentPrefix = 0;
    int count = 0;

    for (int num : nums) {
        currentPrefix += num;
        count += prefixCount.getOrDefault(currentPrefix - k, 0);
        prefixCount.put(currentPrefix, prefixCount.getOrDefault(currentPrefix, 0) +
1);
    }

    return count;
}
```

Why prefixCount.put(0, 1)? This handles subarrays that start at index 0. If currentPrefix itself equals k, then currentPrefix - k = 0, and we need that 0 to already be in the map so it gets counted.

Trace through nums = [1, 2, 3] with k = 3:

i	num	currentPrefix	currentPrefix - k	Map lookup	count	Map state after
-	-	0	-	-	0	{0:1}

i	num	currentPrefix	currentPrefix - k	Map lookup	count	Map state after
0	1	1	1 - 3 = -2	0	0	{0:1, 1:1}
1	2	3	3 - 3 = 0	1	1	{0:1, 1:1, 3:1}
2	3	6	6 - 3 = 3	1	2	{0:1, 1:1, 3:1, 6:1}

Result: 2 subarrays sum to 3. They are [1, 2] (indices 0-1) and [3] (index 2).

Time O(n), space O(n). This problem combines prefix sums (this section) with HashMap lookup optimization (Section 5). It is one of the most commonly tested medium-difficulty problems.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 26: Worked Example: Rotate Array

Problem: Rotate Array (LeetCode 189)

Given an integer array, rotate it to the right by k steps. Do it in place with O(1) extra space.

Three Approaches

Approach	Time	Space	Drawback
Extra array: copy to new positions	O(n)	O(n)	Violates O(1) space requirement
Cyclic replacement: move each element to final position	O(n)	O(1)	Complex to implement, tricky GCD cycle logic
Reverse trick: three reversals	O(n)	O(1)	Clean, preferred in interviews

The Reverse Trick

Observation: rotating right by k is equivalent to taking the last k elements and moving them to the front. Three reversals achieve this:

1. Reverse the entire array
2. Reverse the first k elements
3. Reverse the remaining n - k elements

```
public void rotate(int[] nums, int k) {
    int n = nums.length;
    k = k % n;

    reverse(nums, 0, n - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, n - 1);
}

private void reverse(int[] nums, int left, int right) {
    while (left < right) {
```

```
            int temp = nums[left];
            nums[left] = nums[right];
            nums[right] = temp;
            left++;
            right--;
        }
    }
```

Why k = k % n? If k equals n, the array rotates back to its original position. If k is larger than n, only the remainder matters. Without this normalization, the code breaks when k >= n.

Trace through [1, 2, 3, 4, 5, 6, 7] with k = 3:

Step	Operation	Array State
Original	-	[1, 2, 3, 4, 5, 6, 7]
After reverse(0, 6)	Reverse entire array	[7, 6, 5, 4, 3, 2, 1]
After reverse(0, 2)	Reverse first 3	[5, 6, 7, 4, 3, 2, 1]
After reverse(3, 6)	Reverse remaining 4	[5, 6, 7, 1, 2, 3, 4]

The last 3 elements [5, 6, 7] have moved to the front, and the first 4 elements [1, 2, 3, 4] have shifted to the back. Exactly a right rotation by 3.

Why This Approach Wins in Interviews

The reverse trick is preferred because it is clean to implement, easy to explain, and avoids the tricky cycle-counting logic of the cyclic replacement approach. The reverse helper function is a reusable building block. When presenting this, you can say: "I'll use three reversals. The intuition is that reversing the whole array puts the last k elements at the front but in the wrong order, and the two partial reversals fix the ordering."

Time O(n), space O(1). Each element is moved exactly twice (once per reversal it participates in).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 27: Wrong vs Right: Array Mistakes

Mistake 1: Creating a new array when the problem says "in-place"

Wrong: "I'll create a result array, copy elements to their correct positions, and return it." This ignores the O(1) space constraint. Interviewers treat this as failing to read the problem requirements.

Better: check whether the problem specifies "in-place" or "O(1) extra space" before choosing your approach. If it does, use swap-based or read/write pointer techniques.

Mistake 2: Nested loops for range sums

Wrong: computing sum(l, r) by iterating from l to r every time, giving O(n) per query and O(n * q) for q queries.

Better: build a prefix sum array in O(n), then answer each query in O(1). Recognize "range sum" or "subarray sum" as the signal for prefix sums.

Mistake 3: Initializing Kadane's maxSum to 0

Wrong: setting globalMax = 0 before the loop. For an all-negative array like [-3, -2, -1], this returns 0, implying an empty subarray is valid when the problem requires at least one element.

Better: initialize both currentMax and globalMax to nums[0]. This guarantees the result is always an actual subarray sum from the input.

Mistake 4: Off-by-one errors in prefix sum queries

Wrong: computing sum(l, r) as prefix[r] - prefix[l] instead of prefix[r] - prefix[l - 1]. This excludes the element at index l from the sum.

Better: write the formula as prefix[r] - prefix[l - 1], and handle the l == 0 edge case separately (just return prefix[r]).

Mistake 5: Forgetting edge cases

Wrong: not handling empty arrays, single-element arrays, or arrays where all elements are identical.

Better: test your solution mentally on an empty array, a single-element array, and a uniform array before writing code. Most array functions need a length check or a base-case guard at the top.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 28: How to Present Array Solutions in an Interview

Presenting In-Place Solutions

The most important thing to communicate is the invariant: what is true about the elements before the write pointer, and what is true about the elements after it.

Example statement: “I’ll use a read/write pointer approach. Everything to the left of the write pointer is the final result. The read pointer scans forward, and whenever it finds a valid element, it copies it to the write position and advances the write pointer. This processes the entire array in one pass with O(1) extra space.”

Interviewers look for you to articulate WHY the approach works, not just what it does. Stating the invariant demonstrates understanding, not just memorization.

Presenting Prefix Sums

How to introduce the technique: “I can precompute a prefix sum array in O(n) so that any range sum query becomes O(1). Instead of iterating from l to r every time, I use the formula prefix[r] minus prefix[l minus 1].”

When the problem involves counting subarrays with a specific sum, bridge to the HashMap optimization: “Since I need to find how many previous prefix sums equal currentPrefix minus k, I’ll store prefix sum frequencies in a HashMap for O(1) lookup.”

Presenting Kadane’s Algorithm

Mention the algorithm by name. Interviewers recognize it and it signals fluency: “This is the maximum subarray problem, which I’ll solve with Kadane’s algorithm in O(n) time and O(1) space.”

Explain the decision at each step: “At every position, I decide whether to extend the current subarray or start a new one. If the running sum is negative, it would only hurt any future subarray, so I start fresh.”

General Array Interview Guidance

Before coding, always ask: “Can I modify the input array, or do I need to preserve it?” This is a signal of experience. Some problems allow in-place modification, others require the original array for later use.

State both time and space complexity unprompted. For array problems, space complexity is often the differentiator between approaches.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 29: Interview Practice: Arrays

Practice Prompt

“Given an array of integers, find the contiguous subarray with the maximum sum.”

This is LeetCode 53, Maximum Subarray. It is one of the most commonly asked array problems across all companies.

Expected Walkthrough

Step 1, Understand: The array can contain both positive and negative integers. We need a contiguous subarray (not subsequence). The result is the sum, not the subarray itself. Edge case: what if the array has only one element? Then that element is both the max and the answer.

Step 2, Brute Force: Check every possible subarray. Two nested loops, compute sum for each. Time O(n^2), space O(1). This works but is slow for large inputs.

Step 3, Optimize: Kadane’s algorithm. At each position, decide: extend or start fresh. Track the global maximum across all positions. Time O(n), space O(1).

Step 4, Implement: Write the Kadane’s solution from Slide 24. Initialize from nums[0].

Step 5, Verify: Trace through [-2, 1, -3, 4, -1, 2, 1, -5, 4] as shown in Slide 24B. Confirm the answer is 6. Test with all-negative array [-3, -2, -1] to confirm answer is -1, not 0.

Common Fresher Mistakes on Array Problems

Mistake	Why It Happens	What To Do Instead
Modifying input without asking	Assumes modification is allowed	Ask “can I modify the input array?” before coding
Skipping empty/single-element cases	Focused on main logic, forgets boundaries	Test edge cases before coding: empty, single element, all same
Nested loops when linear scan exists	Does not recognize prefix sum or hash map signal	If inner loop is searching for a match, consider hash map. If inner loop computes a range sum, consider prefix sum
Using ArrayList when int[] suffices	Java habit from non-performance code	Use primitive int[] for fixed-size arrays. ArrayList has autoboxing overhead

Interviewer Follow-ups to Prepare For

“Can you also return the starting and ending indices of the maximum subarray?” Track indices where globalMax updates.

“What if I asked for the minimum subarray sum?” Same algorithm, flip Math.max to Math.min.

“What about maximum product subarray?” Different recurrence: track both max and min at each position because a negative times a negative can become the max.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 30: Section Cheat Sheet

Array Operation Costs

Operation	int[]	ArrayList
Access by index	O(1)	O(1)
Search (unsorted)	O(n)	O(n) via .contains()
Append	N/A (fixed size)	O(1) amortized
Insert at index i	Manual shift O(n)	O(n) via .add(i, val)
Delete at index i	Manual shift O(n)	O(n) via .remove(i)

Pattern Recognition Signals

Problem Signal	Pattern	Expected Complexity
“in place”, “O(1) extra space”	Swap or read/write pointer	O(n) time, O(1) space
“range sum”, “subarray sum”	Prefix sum	O(n) build, O(1) query
“subarray sum equals k”, “count subarrays”	Prefix sum + HashMap	O(n) time, O(n) space
“maximum contiguous subarray”	Kadane’s algorithm	O(n) time, O(1) space
“sorted array, find pair”	Two pointers (Section 6)	O(n) time, O(1) space
“contiguous subarray with condition”	Sliding window (Section 7)	O(n) time

Key Formulas

```
Prefix sum build:    prefix[i] = prefix[i - 1] + nums[i]
Prefix sum query:    sum(l, r) = prefix[r] - prefix[l - 1]   (l > 0)
                     sum(0, r) = prefix[r]

Kadane's recurrence:  currentMax = max(nums[i], currentMax + nums[i])
                      globalMax = max(globalMax, currentMax)
                      Initialize both from nums[0], not 0

Rotation:             k = k % n
                      reverse(0, n-1), reverse(0, k-1), reverse(k, n-1)
```

Common Interview Mistakes

Mistake	Wrong	Better
New array when in-place required	“I’ll create a result array”	Use read/write pointers or swap
Nested loops for range sums	O(n) per query, O(n*q) total	Prefix sum: O(n) build, O(1) per query
Kadane’s init to 0	Returns 0 for all-negative arrays	Initialize from nums[0]
Prefix sum off-by-one	prefix[r] - prefix[l]	prefix[r] - prefix[l - 1], handle l == 0
Ignoring hidden O(n) in loop body	“O(n) because one loop” but body has .contains()	Analyze body cost: O(n) loop * O(n) body = O(n^2)

How This Section Connects to Other Sections

Section	Connection
Hash Maps (Section 5)	Subarray Sum Equals K combines prefix sums with HashMap lookup
Two Pointers (Section 6)	Same-direction read/write pointer is the formal version of in-place array techniques
Sliding Window (Section 7)	Operates on arrays/strings for “longest/shortest subarray with condition” problems
Binary Search (Section 8)	Operates on sorted arrays. Array operation costs from this section are prerequisite
Sorting (Section 9)	Sort-then-solve uses O(n log n) preprocessing to simplify array problems

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina